Payment Fraud Detection POC

In today's digital payments landscape, fraud tactics evolve rapidly, rendering static, rule-based systems increasingly ineffective. Relying solely on rigid logic frequently leads to missed fraud patterns or high false-positive rates that alienate good customers.

In this Proof of Concept (PoC) I have to build a modern distributed, automated system to identify and intercept fraudulent transactions while keeping legitimate payments seamless.

Understanding the Problem

In a digital payments platform, every second, customers initiate payments using cards, bank transfers, or digital wallets. Most payments are legitimate, but some are fraudulent. For example:

- A customer's account is compromised, and a fraudster initiates a high payment.
- A payment is made from a new device or new location, followed by several rapid transactions.
- A transaction looks unusual in isolation but is legitimate when viewed against the customer's normal spending pattern.

The payment platform must decide what to do with each payment before the business incurs an unacceptable level of risk.

The goal of this POC is to build a reliable, low-latency payment fraud decisioning system that detects suspicious transactions, prevents likely fraud, minimizes false declines of legitimate payments, and provides an explainable record of every decision.

Breaking the main problem into business sub-problems

The main payment fraud detection problem is broken into the following subproblems. Each one addresses a distinct business need and should map to a clear part of the system architecture.

i) Reliable payment intake: To ensure exactly-once payment processing without missing or duplicating any events.

ii) Detect known fraud patterns: The system should catch obvious fraud quickly and consistently

iii) Understand customer context: To distinguish customer's legitimate behavior from potentially fraudulent behavio**r**

iv) Investigate ambiguous cases: To evaluate transactions that simple deterministic rules cannot confidently classify.

v) Preserve decisions and evidence: Evidence should be stored so that fraud analysts and auditors can understand what happened.

vi) Handle failures safely: What happens when a service, dependency, or event fails

vii) Measure and improve outcomes: To assess the system is accurate, fast, reliable, and worth operating

Designing Reliable payment intake and failure handling

For the payment fraud detection POC, reliable payment intake solves the first critical business problem. Every payment event must reach the fraud decisioning pipeline reliably, at sufficient throughput, without losing business-critical information.

There is a need for an intake architecture that separates payment submission from downstream processing and supports recovery when components fail.

Why choose Kafka?

Kafka is a good choice because payment fraud detection platform is an event-driven system with multiple independent consumers, potentially high throughput, and a need to recover and replay events.

Durable event storage - Kafka retains published events according to its retention policy, so consumers can recover and replay events after an outage.

Horizontal scalability - Partitions allow multiple consumers to process different portions of the payment stream in parallel.

Failure recovery - Consumers can restart and resume from committed offsets, subject to retention and the chosen offset-reset policy.

As part of this POC Kafka topics are divided into 3 partitions for parallel processing and scalability.

Choosing customer_id as the key: Payments from the same customer are routed to the same partition.Their order is preserved in the partition and customer-specific velocity can be coordinated more easily.

Idea behind different consumer groups: The fraud detection application has multiple **independent functions**. Example: Payment processing, Rule and LLM based fraud analysis, audit logging and persistence storage.

These functions generally need to process the same **payment events independently** since these roles are not interchangeable.

Different consumer groups handling different business functions will provide independent consumption progress and scaling.

Failure Handling Safely:

Reliable intake also means handling events that repeatedly fail. **Examples include** malformed JSON, invalid fields, unsupported schema versions, or persistent processing errors.

Transient failures - Retry with bounded backoff.

Permanent or poison events - Publish the original event and error details to a suitable DLQ.

Dead Letter Queue (DLQ) is a safety mechanism for isolating payment events that cannot be processed successfully after an **appropriate retry policy**.

The business objective is not simply to move failed messages somewhere else. It is to ensure that a problematic payment does not silently disappear, repeatedly block processing, or cause an incorrect fraud decision.

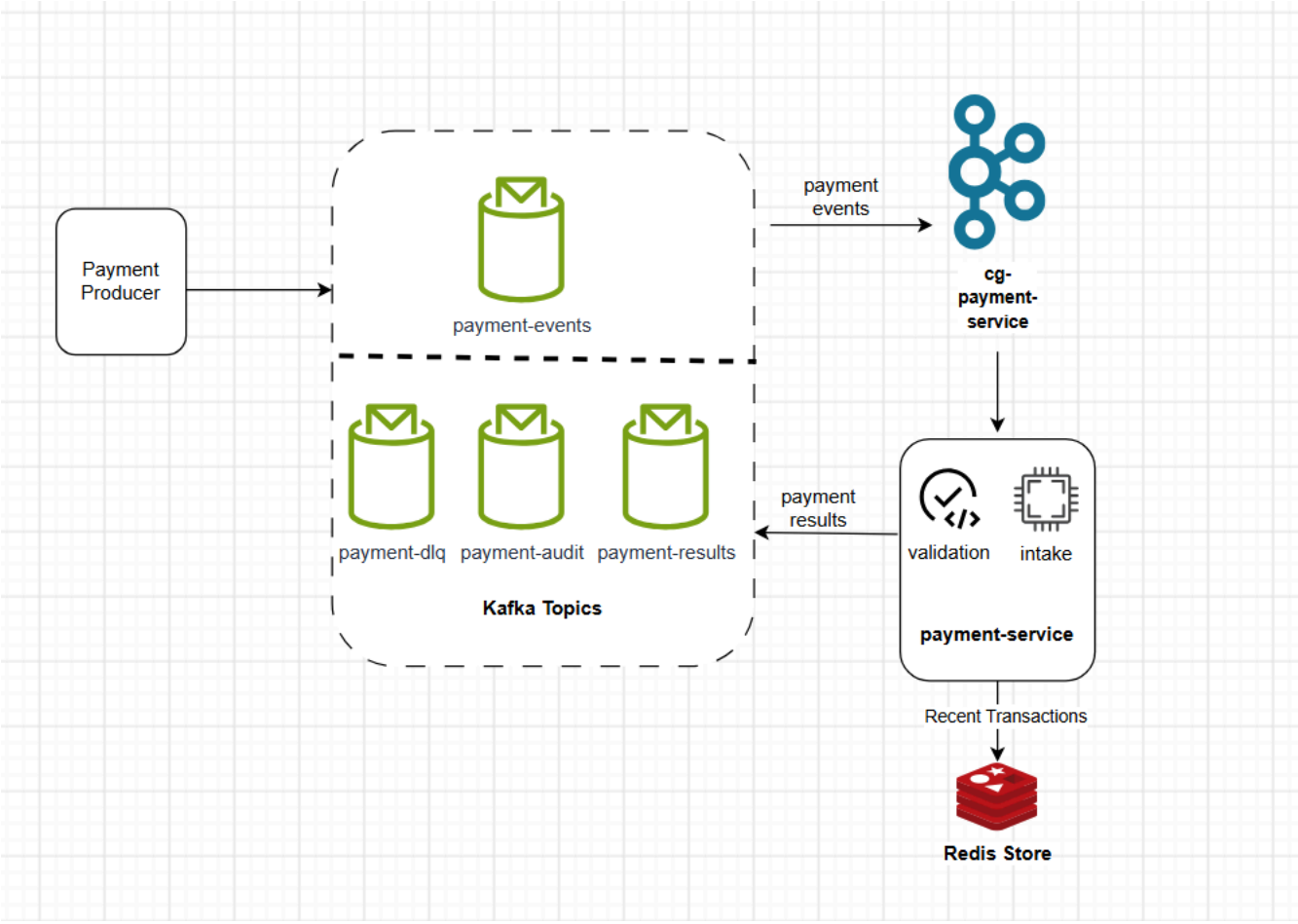
Recent Transactions:

The **recent successful** processed payments are also stored in **Redis cache.** During the later phase of the design this will serve as **valuable context on user behavior** for the LLM based fraud detection service.

Payment Ingestion Flow:

1. As part of POC a simulated producer service produces payment events and publishes them in the **“payment-events”** topic.

2. A separate service **“payment-service”** consumes the payment events using consumer group **“cg-payment-service”**
3. The “payment-service” validates the payment event and after successful processing publishes it into a **“payment-results”** topic which will be later consumed by the **fraud service.**
4. Also the recent n transactions are saved in **Redis cache** to be later used by **LLM based fraud detection service.**
5. The “payment-service” also publishes the payment results in topic **“payment-audit”.** The **kafka-connect** will export these results in the Elasticsearch engine for appropriate **audit logging.**
6. In case of validation failure or reaching retry limit payment-service will send the failed payment events to DLQ topic **“payment-dlq”.**



Designing known fraud patterns detection and audit logging

A payment company needs to identify known suspicious patterns before fraud causes financial losses. Some patterns can be detected with simple, deterministic logic, without waiting for an LLM.

For example, imagine a fraudster has obtained a customer's payment credentials. They may attempt many **transactions quickly**, use the credentials from an **unfamiliar country, or make an unusually large purchase.**

The fraud detection service should evaluate these patterns consistently and produce explainable evidence.

Velocity rule: This rule will measure customer or payment being used unusually frequently within a short period. Example rule: more than 5 payment attempts in 60 seconds.

This will help in fast detection of suspicious bursts, before an attacker can execute many payments.

New-country rule: This rule will evaluate current transactions originating from a country that is unfamiliar or inconsistent with the customer's recent activity. Example: a customer has normally made payments from India, but a new payment appears to originate from another country within a short interval.

High-amount rule: A single fraudulent high-value transaction can create substantial losses. It provides a useful initial signal for deciding whether additional contextual analysis is needed.

A fixed threshold alone is not enough and combining a configurable absolute threshold with customer history, where available.

The above mentioned rules and signals at individual level are not full proof of fraud. **Together, they provide stronger reasons to investigate.**

The Rule Manager should execute the rules and produce structured results and a **decision policy** with a consistent set of evidence **should evaluate the combined results** and infer whether the payment is fraud or require f**urther investigation by LLM service.**

Choice of Redis for velocity and country history:

Redis supports in-memory operations for frequently accessed customer risk state, making it suitable for low-latency fraud checks.

TTL expiration and sorted-set operations can help maintain recent transaction history and rolling velocity windows.

Also Frequent fraud-state lookups do not need to become repeated historical queries against primary payment databases.

When should the fraud service request further LLM analysis:

We do not want every payment to invoke an LLM. That would increase operating cost, introduce additional latency, and create more opportunities for external service failures.

Instead, **rule based fraud service** should act as the first-stage risk filter, and next **LangGraph-based LLM service** should investigate selected ambiguous cases.

A **decision policy** should suggest an LLM based investigation if deterministic outcomes fail to provide sufficient evidence. For example, only one contextual signal, such as a new country.

Elasticsearch for audit logging:

The support team or fraud investigator needs to find the exact payment, retrieve the decision, inspect the triggered rules, and understand whether an LLM review influenced the outcome.

Elasticsearch is a search and analytics engine designed to index and query structured documents. It is useful when investigators need to search a large volume of audit events by payment ID, customer reference, rule, decision, error, or time range.

Fast investigation - Search for a payment ID or retrieve all decisions that triggered HIGH_VELOCITY during a specified period.

Operational visibility - Analyze trends in blocked payments, review rates, recurring processing failures, and rule triggers.

Structured evidence - Store searchable JSON documents with decision IDs, rule results, timestamps, and relevant processing metadata.

Sample audit log json:


```
{
  "event_id": "audit-9001",
  "payment_id": "P200",
  "customer_id": "C100",
  "event_type": "FRAUD_DECISION",
  "decision": "REVIEW",
  "triggered_rules": [
    "HIGH_VELOCITY",
    "NEW_COUNTRY"
  ],
  "llm_invoked": true,
  "reason": "Multiple contextual risk signals",
  "timestamp": "2026-10-23T05:30:00Z"
}
```


Kafka Connect to synchronize audit events into Elasticsearch:

We could write Python code that sends every audit document directly to Elasticsearch. But then **every application would need to manage Elasticsearch connectivity, retries, bulk indexing, delivery failures, and destination-specific behavior.**

Kafka Connect separates those integration responsibilities from the fraud business logic.

Loose Coupling - The fraud service does not need to know Elasticsearch's indexing API or handle every sink-specific concern.

Recoverable delivery - Connector retries and source offsets help resume delivery after recoverable failures, subject to the connector's guarantees and configuration.

Operational separation - Connector configuration, monitoring, retries, and supported dead-letter behavior can be managed separately from fraud decision logic.

Deterministic Payment Fraud Detection Flow:

1. The rule based **“determenstic-fraud-service”** consumes the payment results (discussed previously) from topic **“payment-results”** using consumer group **“cg-fraud-service”**

2. The **rule manager** running inside the “determenstic-fraud-service” applies different rule set like **New country, High Amount and High Velocity** to assess the fraud risk score.

3. To evaluate the rules relevant contexts (like country and transaction velocity) are fetched from **Redis store.**

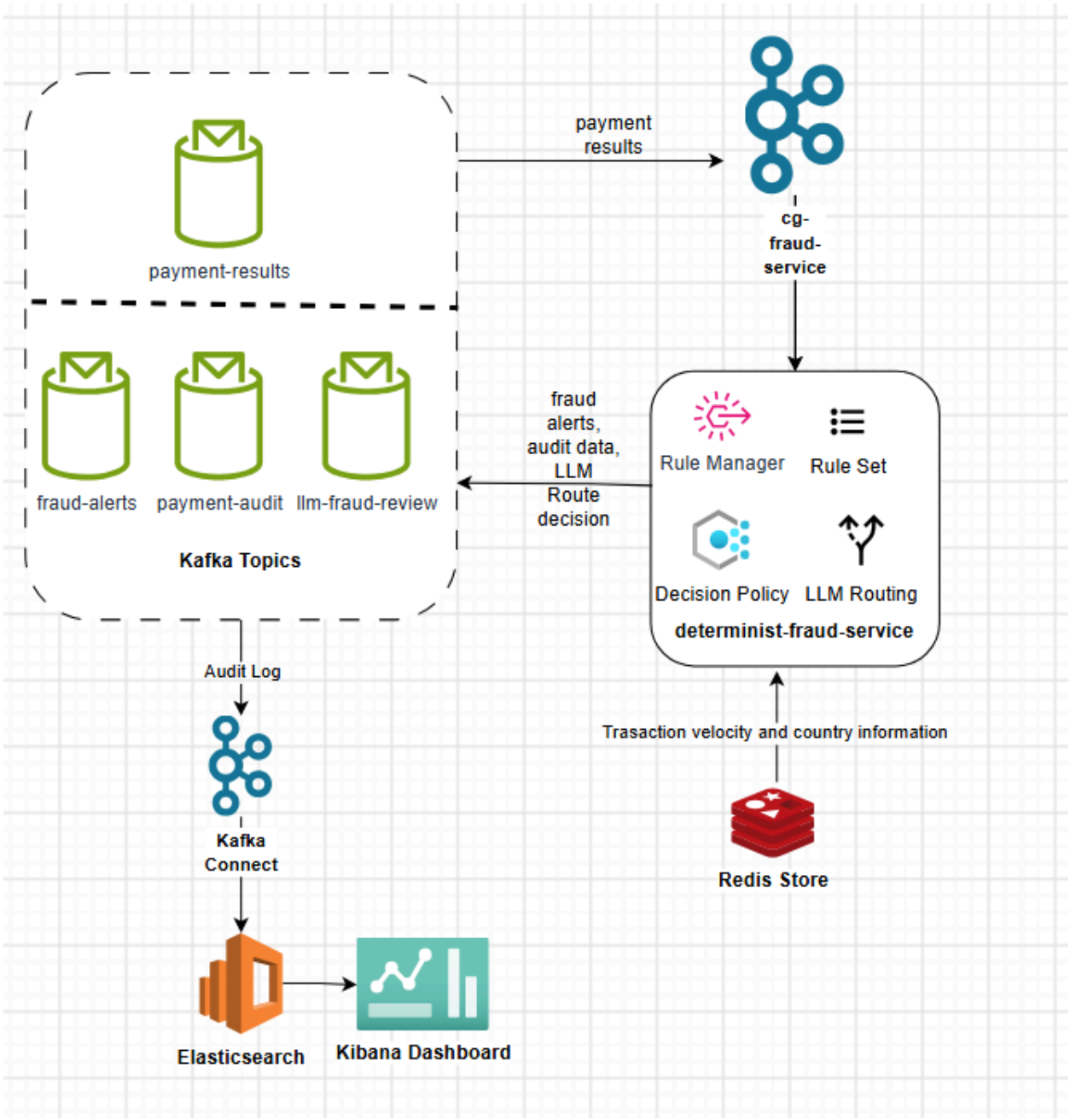
4. Once rules are executed then a **decision policy** is applied combining different rule set results for identifying potential frauds.

5. The identified fraud payment events are sent in topic **“fraud-alerts”** which can be used to generate alerts and notifications to the customer.

6. For ambiguous results the payment event is flagged for further review by LLM based fraud service and sent to topic **“llm-fraud-review”**

7. The **“determenstic-fraud-service”** also publishes the fraud analysis in topic **“payment-audit”** for audit logging.

8. The **kafka-connect** exports the audit log data to **Elasticsearch engine** and the investigator can then analyze them in the **Kibana dashboard.**


payment-results
payment results
cg-fraud-service
fraud-alerts
payment-audit
llm-fraud-review
Kafka Topics
fraud alerts, audit data, LLM Route decision
Rule Manager
Rule Set
Decision Policy
LLM Routing
determinist-fraud-service
Audit Log
Trasaction velocity and country information
Kafka Connect
Redis Store
Elasticsearch
Kibana Dashboard

Designing ambiguous cases investigation through LLM based fraud detection service

A hybrid approach has been used in the fraud detection system: Deterministic rules handle known fraud patterns, while an **LLM-based agent** investigates suspicious or ambiguous transactions that need deeper contextual analysis.

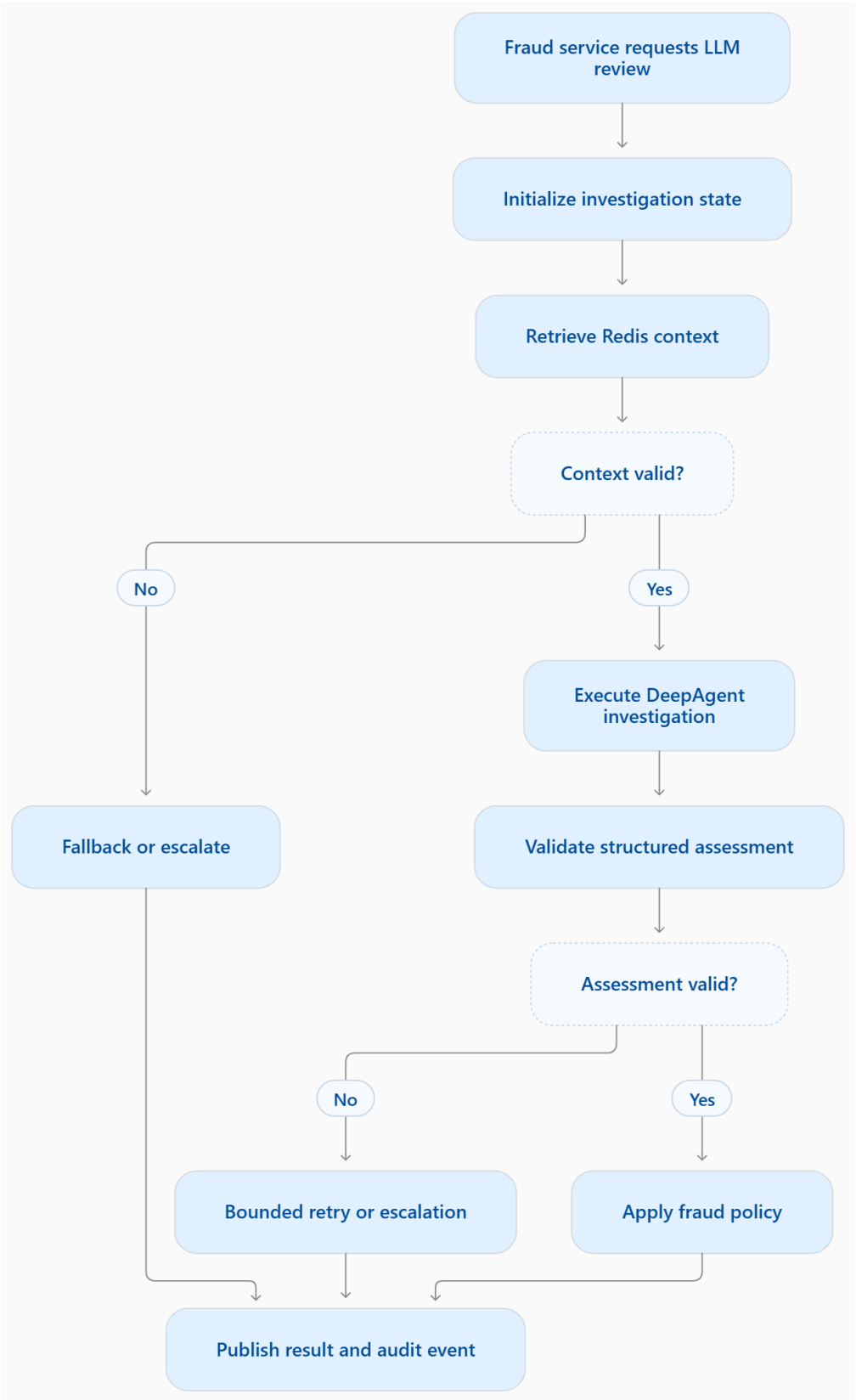
LangGraph as the control plane for orchestration:

A suspicious transaction may require several dependent operations:

1. Retrieve the customer's profile.
2. Retrieve the customer's last five transactions.
3. Determine which evidence is relevant.
4. Ask an LLM to assess the risk.
5. Validate the output.
6. Decide whether to accept the assessment, retry, or escalate.

So there is a need for a component that controls the order of these operations and manages what happens when one fails.

LangGraph should **orchestrate the investigation**, not grant the LLM unrestricted authority to approve or decline payments. The final payment action should remain governed by **explicit fraud policies.**


Fraud service requests LLM review
Initialize investigation state
Retrieve Redis context
Context valid?
No
Yes
Execute DeepAgent investigation
Fallback or escalate
Validate structured assessment
Assessment valid?
No
Yes
Bounded retry or escalation
Apply fraud policy
Publish result and audit event

Langfuse for observability:

As part of observability we need to investigate and monitor which context was retrieved, which prompt was used, what tools were called, what the model returned, how long each operation took, and whether the final assessment was correct.

End-to-end traces - Trace an individual fraud investigation across context retrieval, agent execution, tool calls, model calls and final validation.

Latency and performance - Measure total investigation latency and the time spent on Redis, individual tools and model calls.

Token usage and cost - Track prompt and completion token usage and model costs where supported by the integration.

Prompt and tool debugging - Inspect prompt versions, model outputs, tool execution results and errors, subject to your tracing configuration.

Flow of LLM based fraud analysis:

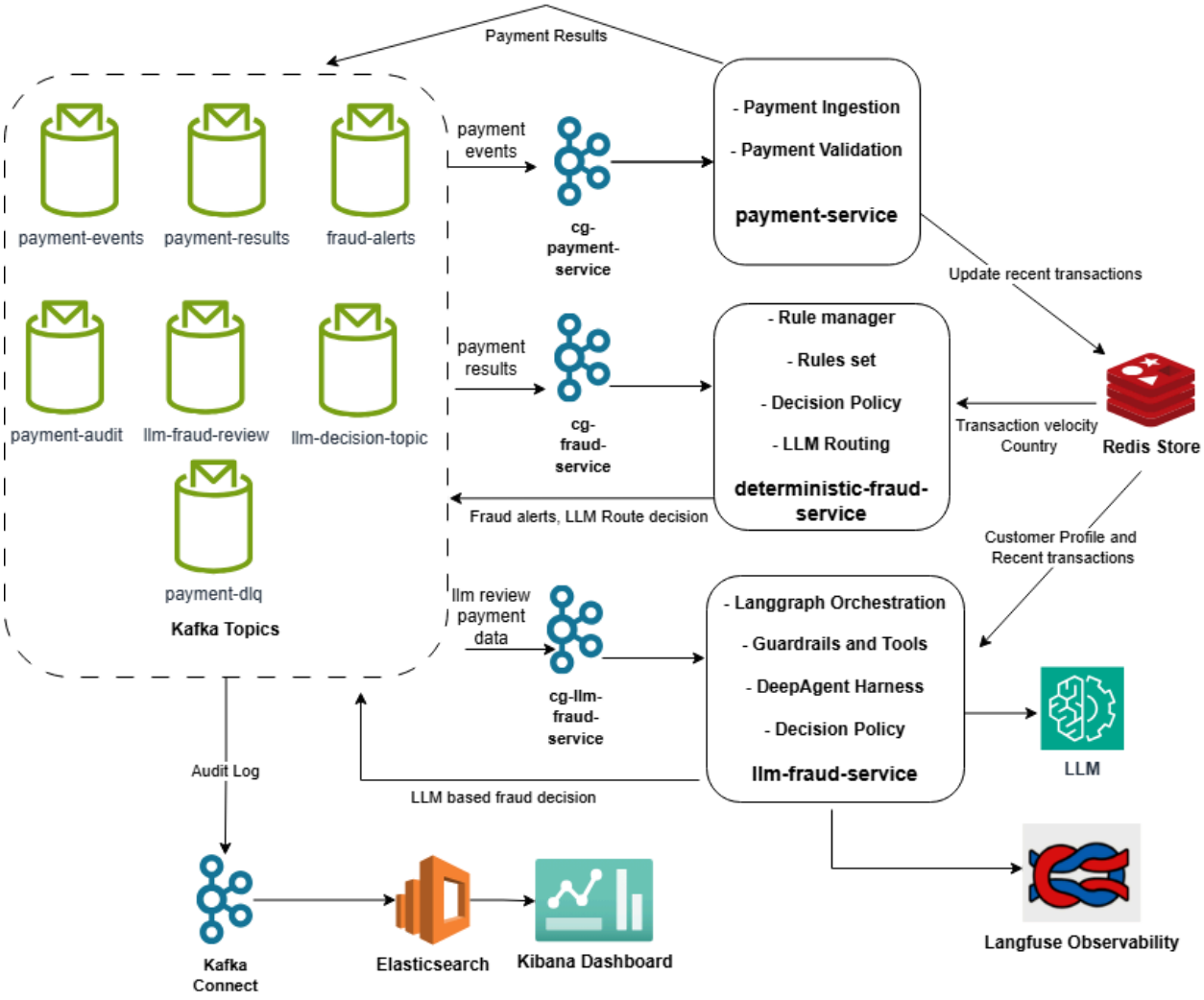
1. The LLM based fraud service **“llm-fraud-service”** consumes the payment analysis results done by rule based service from topic and flagged for llm review. It uses consumer group **“cg-llm-fraud-service”** to retrieve the data from topic **“llm-fraud-review”**

2. The **Langgraph** orchestrates the analysis and applies different **guardrails** for validating input and LLM outputs. It also maintains a safe fallback step in case of any exception during execution of agent loop.

3. The customer contexts like profile and last 5 transactions are retrieved from **Redis store** through **agent tools.** Along with that the **rule based fraud analysis result** is also used as context for the LLM

4. The execution layer is delegated to **langchain deepagent** and it plays the role of agent harness. Here the agent can be run in a restricted manner by configuring a proper **harness profile.**

5. The agent analysis is then further investigated by a **decision policy** which makes the final decision.


Payment Results
payment-events
payment-results
fraud-alerts
payment-audit
llm-fraud-review
llm-decision-topic
payment-dlq
Kafka Topics
payment events
cg-payment-service
- Payment Ingestion
- Payment Validation
payment-service
Update recent transactions
payment results
cg-fraud-service
- Rule manager
- Rules set
- Decision Policy
- LLM Routing
deterministic-fraud-service
Transaction velocity
Country
Redis Store
Fraud alerts, LLM Route decision
Customer Profile and Recent transactions
llm review payment data
cg-llm-fraud-service
- Langgraph Orchestration
- Guardrails and Tools
- DeepAgent Harness
- Decision Policy
llm-fraud-service
LLM
Audit Log
LLM based fraud decision
Kafka Connect
Elasticsearch
Kibana Dashboard
Langfuse Observability